

Common Lab Mistakes

EE5203 Microprocessors — what goes wrong, what you see, and how to fix it

Basri Erdogan — 2026-2027 Fall

Almost every lost laboratory hour comes from one of the mistakes below. None of them is a coding mistake. They are all about **which folder you opened**, **where the .vscode folder sits**, and **which button you pressed**.

Use this page in two ways: read it once before Lab 1, and search it by symptom when something fails.

Check these three things first

Before you ask for help, look at these three:

1. **Title bar of VS Code.** It must show your project folder — the folder that contains the .ioc file. Not the folder above it.
2. **Explorer panel, top level.** You must see .vscode, Core, Drivers, CMakeLists.txt, <name>.ioc side by side.
3. **.vscode contents.** Five files: tasks.json, launch.json, settings.json, fix_ld.cmake, STM32F401RETx_FLASH.ld.

If all three are correct, the problem is in your code and you can debug it. If any one is wrong, fix it first — the error message you are reading is probably a side effect.

Symptom lookup

What you see	Go to
F5 asks you to “Select debugger” / “Select environment”	1 , 4 , 5
Ctrl+Shift+B says “No build task to run found”	1 , 4 , 5
CMake Error: Could not read presets / “No such preset”	1 , 8
STM32F401RETx_FLASH.ld:56: syntax error	12 , 5
Unable to start debugging. Program file ...	7 , 13
.elf does not exist	
cmake -P ... fix_ld.cmake → file not found	4 , 5
Compiler errors about iostream, cl.exe, g++	14
ST-LINK GDB server error, target busy, cannot connect	15
CubeMX keeps offering to download STM32CubeF4	10 , 11
Your code is gone after you changed a pin in CubeMX	17

Opening the project

1. You opened the folder *above* the project

What you did: File → Open Folder → C:\Work\2400001234, the folder that holds all your projects.

What you see: No preset question. Ctrl+Shift+B reports “No build task to run found”. F5 asks you to pick a debugger. CMake Tools cannot find CMakePresets.json.

Why: VS Code reads .vscode, CMakeLists.txt and CMakePresets.json only at the **top level of the folder you opened**. One level up, they are invisible to it.

Fix: File → Open Folder → the folder that contains the .ioc file, for example C:\Work\2400001234\Blink. Check the title bar.

2. You opened a folder *inside* the project

What you did: Opened Core, Core\Src, or build.

What you see: The same symptoms as mistake 1. `main.c` opens and edits normally, which makes the mistake easy to miss.

Why: Same reason. The project files are now *above* the folder you opened.

Fix: Open the project folder itself. Being able to edit `main.c` proves nothing — any folder lets you edit files.

3. You opened a file, not a folder

What you did: Double-clicked `main.c`, or used File → Open File.

What you see: VS Code says “**You have not yet opened a folder**”. Nothing builds.

Why: Without an open folder there is no workspace, so there are no tasks and no debug configuration.

Fix: File → Open Folder. On the laboratory PCs, start from the **VS Code - EE Labs** shortcut and open the folder from there.

The .vscode template

4. You copied the five files instead of the folder

What you did: Opened `C:\Users\Public\EE5203_template\.vscode`, selected the files inside, and pasted them into your project.

What you see: `tasks.json`, `launch.json` and the rest sit loose next to `main.c` and `CMakeLists.txt`. `Ctrl+Shift+B` finds no task; `F5` asks for a debugger.

Why: VS Code only reads these files from a folder named exactly `.vscode`. Loose in the project root they mean nothing.

Fix: Go one level up, copy the **folder** `.vscode` itself, and paste it into your project folder. Then delete the loose copies.

5. The .vscode folder is in the wrong place

What you did: Pasted `.vscode` into `C:\Work\2400001234` (next to the project), or into `Core`, or into `build`.

What you see: A mixture: no build task, or the build runs but stops at `STM32F401RETx_FLASH.ld:56: syntax error`, or `cmake` reports it cannot find `fix_ld.cmake`.

Why: The repair script looks for the linker script in the folder directly above itself. Put one folder off, it repairs the wrong folder — or nothing.

Fix: `.vscode` belongs **inside the project folder, next to the `.ioc` file:**

```
C:\Work\2400001234\Blink\  
  .vscode\           <- here  
  Core\  
  Drivers\  
  CMakeLists.txt  
  Blink.ioc
```

6. You renamed .vscode

What you did: Windows hid the leading dot, or you saved it as `vscode`, `.vscode (2)`, or `vscode-template`.

What you see: As if the folder were missing.

Fix: The name must be exactly `.vscode`, with the dot, no space, no number.

7. You renamed the project folder after generating it

What you did: CubeMX generated `Blink`, and you renamed the folder to `Lab1`.

What you see: The build succeeds, then debugging fails with “**Program file does not exist**” — it is looking for `build\Debug\Lab1.elf` while the build produced `Blink.elf`.

Why: The debug configuration takes the executable name from the **folder name**; CMake takes it from the **CubeMX project name**. Renaming breaks the match.

Fix: Choose the name in CubeMX, before generating, and do not rename afterwards. If you already did, rename the folder back.

Creating the project in CubeMX

8. Toolchain/IDE is not CMake

What you did: Left Project Manager → Toolchain/IDE at **STM32CubeIDE**, or chose Makefile or EWARM.

What you see: There is no `CMakePresets.json` and no usable `CMakeLists.txt`. VS Code reports it cannot read presets, and nothing builds.

Fix: Project Manager → Toolchain/IDE → **CMake** → Generate Code again. Generate into a **new, empty folder**; do not mix two toolchains in one folder.

9. Bad project location or project name

What you did: Saved to the Desktop, to H:, to a OneDrive folder, or used a name with spaces or Turkish characters (Ölçüm 1, Deney Bir).

What you see: Unpredictable build failures, paths reported as not found, very slow builds, or files that vanish between sessions.

Why: The build tools pass these paths to the compiler as plain text. Spaces and non-ASCII characters break them, and network drives are too slow and too unreliable for a build.

Fix: Generate into `C:\Work\<studentnumber>`. Project names: letters, digits and underscore only.

10. You accepted the STM32CubeF4 download

What you did: Clicked **Yes / Download** when CubeMX offered the STM32CubeF4 package.

What you see: A long download that may fail, and a second copy of the package in your own user folder.

Why: The package is already installed on the laboratory PC, in a shared folder.

Fix: Cancel. Help → **Updater Settings** → Repository Folder: `C:/Users/Public/STM32Cube/Repository` → OK → restart CubeMX.

11. You browsed *into* the package folder

What you did: In Updater Settings you browsed down to `...\Repository\STM32Cube_FW_F4_V1.28.2`.

What you see: CubeMX still offers to download the package.

Why: The setting expects the folder that **contains** the packages, not one package.

Fix: Set it to `C:/Users/Public/STM32Cube/Repository` — type it, do not browse into a subfolder.

Building and running

12. You used the status bar Build button

What you did: Pressed the **Build** button in the blue status bar, or ran *CMake: Build* from the Command Palette.

What you see: `STM32F401RETx_FLASH.ld:56: syntax error and no .elf`.

Why: CubeMX has a known bug: the linker script it generates loses its memory region names. The build task in `.vscode` repairs the file first. The status bar button runs CMake directly and skips that repair. CubeMX breaks the file again on every regeneration.

Fix: Always build with **Ctrl+Shift+B** (Terminal → Run Build Task).

13. You picked the Release preset

What you did: When CMake Tools asked for a configure preset, chose **Release** or **RelWithDebInfo**.

What you see: The build succeeds, but debugging reports the `.elf` does not exist — the debug configuration looks in `build\Debug\`.

Fix: Choose **Debug**. To change it: Command Palette → *CMake: Select Configure Preset* → Debug, then Ctrl+Shift+B.

14. You pressed the `Run` in the editor title bar

What you did: Clicked the play triangle at the top right of the `main.c` editor, or chose *Run C/C++ File*.

What you see: A prompt to choose a debugger, or compiler errors mentioning `cl.exe`, `g++`, or missing standard headers.

Why: That button tries to compile and run the single file **on the PC**. Your program is for the STM32, not for Windows.

Fix: Press **F5**, or use Run and Debug view → next to **Debug (ST-LINK)**. F5 builds, flashes and starts debugging in one step; there is no separate “program the board” button.

15. Something else is holding the ST-LINK

What you did: Left a debug session running, opened a second VS Code window on the same project, or left STM32CubeProgrammer connected.

What you see: ST-LINK GDB server errors, “cannot connect to target”, or the debugger starting and dying immediately.

Why: Only one program can use the ST-LINK at a time.

Fix: Stop the previous debug session (Shift+F5), close extra windows and CubeProgrammer, unplug and replug the board, then F5 again. The first F5 after replugging sometimes fails on its own; a second F5 usually works.

16. You answered the “install cube bundle(s)” notification

What you did: Clicked the notification that offers to install ST bundles.

What you see: F5 waits and never starts the debugger.

Why: The extension blocks debugging until the question is answered, and the lab PCs already have everything.

Fix: Answer **No**, or dismiss it. If F5 is stuck, check the bell icon in the bottom right for an unanswered question.

Keeping your work

17. You wrote code outside the USER CODE regions

What you did: Typed statements between `/* USER CODE END WHILE */` and `/* USER CODE BEGIN 3 */`, or anywhere outside a marked region.

What you see: The code works, until you change a pin in CubeMX and regenerate — then it is gone.

Why: Regeneration rewrites every line except those inside `USER CODE BEGIN ... END` pairs.

Fix: Keep every statement you write inside a region. `BEGIN 2` for variables before the loop, `BEGIN 3` for the loop body.

18. You worked directly on H:

What you did: Opened the project from `H:\EE_Labs` and built it there.

What you see: Very slow builds, strange path errors, and sometimes a corrupted `build` folder.

Fix: Copy the project from `H:\EE_Labs` into `C:\Work\<studentnumber>` first, work there, and at the end of the session run the **Save to H** desktop shortcut. Anything left only on `C:` may be gone next week.

Warning: *Save to H* makes `H:\EE_Labs` an exact copy of `C:\Work\<studentnumber>`. Projects that are on `H:` but not on this PC are **deleted**. So at the start of every session copy **all** your projects back from `H:` — not only the one you are working on.

When you still need help

Bring these four lines, written down:

Expected:

Observed:

Evidence collected:

Smallest next test: